The Smallest Enclosing Ball

An undergraduate introduction, and an active-set algorithm that solves it exactly

Thomas Schmelzer

September 3, 2026

Abstract

Given n points in $\mathbb{R}^d$, what is the smallest ball that contains all of them? The question is easy to state, easy to picture, and — once you notice that the answer is pinned down by a mere handful of the points — easy to solve exactly. These notes develop the problem from scratch for a reader who has met linear algebra and a little calculus. We prove that the ball exists and is unique, derive a certificate of optimality that can be checked by inspection, show that the problem is a convex quadratic program in disguise, and then build an active-set algorithm around the certificate. The algorithm is short enough to write on one page, terminates at the exact answer rather than at a tolerance, and costs one tiny dense linear solve per iteration. We close with the numerical details that separate a correct implementation from a fragile one.

Contents

1 The problem

Let $S = \{p_1, \ldots, p_n\} \subset \mathbb{R}^d$ be a finite set of points. Write

$$B(x, r) \;=\; \{\, z \in \mathbb{R}^d \;:\; \|z - x\| \le r \,\}$$

for the closed Euclidean ball with centre x and radius r. We want the *smallest enclosing ball* (SEB) of S: a ball containing every point of S whose radius is as small as possible. Written as an optimisation problem,

$$\begin{aligned} &\text{minimise} && r \\ &\text{subject to} && \|p_i - x\| \le r, \qquad i = 1, \ldots, n, \end{aligned} \tag{1}$$

with decision variables $x \in \mathbb{R}^d$ and $r \in \mathbb{R}$. Equivalently, and more compactly, we are minimising the function

$$f(x) \;=\; \max_{1 \le i \le n} \|p_i - x\| \tag{2}$$

over $x \in \mathbb{R}^d$; the optimal radius is $R = \min_x f(x)$ and the optimal centre is the minimiser. The centre is sometimes called the *Chebyshev centre* of S, and the problem the 1-*centre problem*.

Why anyone cares. The SEB is a small problem that shows up inside larger ones.

- *Bounding volumes.* Collision detection, ray tracing and spatial indexing all want a cheap conservative proxy for a complicated object. A sphere is the cheapest one going: an intersection test is a single distance comparison, and unlike an axis-aligned box it does not change when you rotate the scene.
- *Facility location.* Place one emergency depot so that the *worst-case* travel distance to any of n towns is as small as possible. That is exactly (2).
- *Machine learning.* The support vector data description and the one-class SVM fit a smallest enclosing ball around the training data (in a feature space) and flag anything outside it as an anomaly. The points on the boundary — Section 3 will name them — are the support vectors.
- *Robust statistics and rounding.* The centre is a summary of the cloud that, unlike the mean, is completely insensitive to how many points sit in a dense cluster and completely sensitive to the extremes.

A first, false, idea. The mean $\bar{p} = \frac{1}{n}\sum_i p_i$ does *not* solve the problem. Take $p_1 = (0,0)$, $p_2 = (1,0)$ and then a hundred copies of $(1,0)$: the mean is dragged towards the cluster, while the smallest enclosing ball has not moved at all. The SEB depends only on the shape of the cloud's outer boundary, and — as we will see — on very little of even that.

2 Existence and uniqueness

Before computing anything, we should know there is something to compute.

Proposition 2.1 (Existence)**.** *For any finite non-empty $S \subset \mathbb{R}^d$ the minimum in (2) is attained.*

Proof. Each $x \mapsto \|p_i - x\|$ is continuous, and a maximum of finitely many continuous functions is continuous, so f is continuous. Fix any x_0 and let $\rho = f(x_0)$. Outside the compact set $K = B(x_0, 2\rho)$ we have $f(x) \geq \|p_1 - x\| \geq \|x - x_0\| - \|p_1 - x_0\| > 2\rho - \rho = \rho$, so no point outside K can beat x_0. A continuous function on the compact set K attains its minimum, and that minimum is the global one. □

Uniqueness is the more interesting statement, and the proof contains the geometric idea that the whole algorithm rests on: *two different balls of the same radius can always be replaced by a smaller one.*

Lemma 2.2 (Shrinking lemma)**.** *Let $x_1 \neq x_2$ and let $m = \frac{1}{2}(x_1 + x_2)$, $\delta = \frac{1}{2}\|x_1 - x_2\| > 0$. If $S \subseteq B(x_1, R)$ and $S \subseteq B(x_2, R)$, then $S \subseteq B\big(m, \sqrt{R^2 - \delta^2}\big)$.*

Proof. For any p, the parallelogram law gives the identity

$$\|p - m\|^2 \;=\; \tfrac{1}{2}\|p - x_1\|^2 + \tfrac{1}{2}\|p - x_2\|^2 - \tfrac{1}{4}\|x_1 - x_2\|^2 .$$

(Expand both sides; the cross terms cancel.) If $p \in S$ then both squared distances on the right are at most R^2, so $\|p - m\|^2 \leq R^2 - \delta^2$. □

Theorem 2.3 (Uniqueness)**.** *The smallest enclosing ball of a finite non-empty $S \subset \mathbb{R}^d$ is unique.*

Proof. Let R be the optimal radius and suppose $B(x_1, R)$ and $B(x_2, R)$ both contain S with $x_1 \neq x_2$. By Lemma 2.2, S is contained in a ball of radius $\sqrt{R^2 - \delta^2} < R$, contradicting the minimality of R. □

Geometrically, Lemma 2.2 says that the intersection of two equal balls is a lens, and a lens fits inside a strictly smaller ball centred at its waist. We will meet the same computation again, in disguise, as equation (3).

3 The certificate

Now the key structural fact. Some points of S touch the boundary of the optimal ball, and the rest are strictly inside and completely irrelevant — delete them and the answer does not change. Which points touch, and how many?

Definition 3.1 (Support set)**.** Given a ball $B(x, R)$ containing S, its *support set* is

$$T(x, R) \;=\; \{\, p \in S \;:\; \|p - x\| = R \,\},$$

the points lying exactly on the boundary sphere.

Touching the boundary is not by itself enough to be optimal — a huge ball can have points on its boundary. The extra ingredient is that the support points must *surround* the centre.

Theorem 3.2 (Sufficiency of the certificate)**.** *Suppose $S \subseteq B(x,R)$ and there are points $q_1,\dots,q_m \in T(x,R)$ and weights $w_1,\dots,w_m \geq 0$ with*

$$\sum_{j=1}^{m} w_j = 1 \qquad \text{and} \qquad x = \sum_{j=1}^{m} w_j q_j,$$

that is, $x \in \operatorname{conv} T(x,R)$. Then $B(x,R)$ is the smallest enclosing ball of S.

Proof. Let $y \in \mathbb{R}^d$ be any other candidate centre. Because $\sum_j w_j(q_j - x) = x - x = 0$, expanding about x makes the cross term vanish:

$$\begin{aligned} \sum_{j=1}^{m} w_j \|q_j - y\|^2 &= \sum_{j=1}^{m} w_j \|q_j - x\|^2 + 2(x-y)^{\mathsf{T}} \underbrace{\sum_{j=1}^{m} w_j (q_j - x)}_{=\,0} + \|x-y\|^2 \\ &= R^2 + \|x-y\|^2 . \end{aligned} \tag{3}$$

The left-hand side is a weighted average of the numbers $\|q_j - y\|^2$, so at least one of them is at least as large as the average:

$$\max_{p \in S} \|p-y\|^2 \;\geq\; \max_{j} \|q_j - y\|^2 \;\geq\; R^2 + \|x-y\|^2 .$$

Hence every enclosing ball centred at y has radius at least R, with equality only if $y = x$. □

Equation (3) is worth pausing on: it is the parallel-axis theorem of mechanics (or, if you prefer, the bias–variance decomposition). It says that moving the centre away from the weighted mean of the support points increases the average squared distance by exactly the square of how far you moved. There is nowhere to run.

The converse holds too, so the certificate is not merely sufficient but characteristic.

Proposition 3.3 (Necessity of the certificate)**.** *If $B(x,R)$ is the smallest enclosing ball of S, then $x \in \operatorname{conv} T(x,R)$.*

Proof. Suppose not. The set $C = \operatorname{conv} T(x,R)$ is compact and convex and $x \notin C$, so by the separating hyperplane theorem there is a direction $v \in \mathbb{R}^d$ with $v^{\mathsf{T}}(p - x) > 0$ for every $p \in T(x,R)$. Move the centre along v: for $t > 0$ and $p \in T(x,R)$,

$$\|p - (x + tv)\|^2 = R^2 - 2t\, v^{\mathsf{T}}(p-x) + t^2 \|v\|^2 < R^2$$

once t is small enough. The remaining points satisfy $\|p - x\| < R$ strictly, and there are finitely many of them, so by continuity they also stay strictly inside for all sufficiently small t. For such a t the ball $B(x+tv, R')$ with $R' = \max_i \|p_i - (x+tv)\| < R$ encloses S — contradicting minimality. □

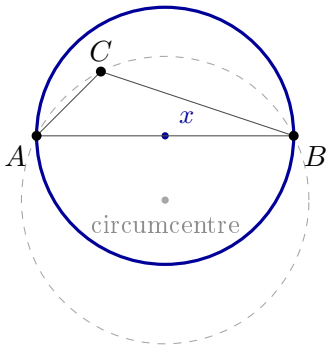
Finally, how large does the support set need to be? Not very.

Corollary 3.4 (At most $d+1$ points matter)**.** *There is a subset $T \subseteq T(x,R)$ with $|T| \leq d+1$ and $x \in \operatorname{conv} T$. Consequently the smallest enclosing ball of S equals the smallest enclosing ball of some subset of at most $d+1$ points.*

Proof. Carathéodory's theorem: a point in the convex hull of a set in $\mathbb{R}^d$ lies in the convex hull of at most $d+1$ of its points. Theorem 3.2 applied to that subset — whose points are still at distance exactly R — shows $B(x, R)$ is optimal for it, and any ball enclosing S encloses the subset, so the two problems have the same answer. □

In the plane this says: the answer is always determined by either two points (a diameter) or three points (a circumcircle). That is a remarkably small amount of information to extract from a cloud of a million points, and it is precisely what the algorithm of Section 5 goes looking for.

Example 3.5 (The circumcircle is often wrong)**.** Take $A = (0, 0)$, $B = (4, 0)$, $C = (1, 1)$. The circle through all three has centre $(2, -1)$ and radius $\sqrt{5} \approx 2.236$ (Exercise 1 asks you to check this). But the smallest enclosing circle has AB as a diameter: centre $(2, 0)$, radius 2, and indeed $\|C - (2, 0)\| = \sqrt{2} < 2$, so C is strictly inside and is not a support point.



This example is the whole difficulty of the problem in miniature. Fitting a circle *through* a chosen set of points is easy linear algebra; the hard part is choosing the right set. Section 5 will show how the arithmetic itself tells us that C has to go.

4 Two convex formulations

Problem (1) is a convex optimisation problem: the objective r is linear and each constraint $\|p_i - x\| \le r$ defines a convex set (a second-order cone, in the joint variable (r, x)). So it can be handed to a general-purpose conic solver, and the package accompanying these notes does exactly that in one of its two solvers. But there is a second formulation that is much closer to the geometry of Section 3, and it is the one we will actually compute with.

4.1 A quadratic program in the weights

The certificate involved weights $w_j \ge 0$ summing to one. Let us make those the variables. Write

$$\Delta_n \;=\; \Big\{\, u \in \mathbb{R}^n \;:\; u_i \ge 0,\ \textstyle\sum_i u_i = 1 \,\Big\}$$

for the unit simplex, and for $u \in \Delta_n$ define the *weighted centre* and the function

$$x(u) \;=\; \sum_{i=1}^{n} u_i p_i, \qquad \varphi(u) \;=\; \sum_{i=1}^{n} u_i \,\|p_i\|^2 \;-\; \|x(u)\|^2 . \tag{4}$$

φ is a *concave* quadratic: the first term is linear in u and the second is a convex quadratic being subtracted. So maximising φ over Δ_n is a convex quadratic program — and Δ_n is about the friendliest feasible set there is.

The link to our problem is a single identity. For *any* $y \in \mathbb{R}^d$ and any $u \in \Delta_n$, expanding as in (3) gives

$$\sum_{i=1}^{n} u_i \, \|p_i - y\|^2 \;=\; \varphi(u) \;+\; \|x(u) - y\|^2 . \tag{5}$$

Proposition 4.1 (Weak duality)**.** *If $B(y, r)$ encloses S then $\varphi(u) \le r^2$ for every $u \in \Delta_n$.*

Proof. Each $\|p_i - y\|^2 \le r^2$, so the left side of (5) is a weighted average of numbers no larger than r^2 and hence at most r^2. The right side is at least $\varphi(u)$. □

So every choice of weights produces a *lower bound* $\sqrt{\varphi(u)}$ on the optimal radius, and every enclosing ball produces an *upper bound*. The certificate of Section 3 is exactly the statement that the two bounds can be made to meet.

Theorem 4.2 (Strong duality)**.** *Let R be the optimal radius. Then*

$$R^2 \;=\; \max_{u \in \Delta_n} \varphi(u),$$

and if w is a maximiser then $x = x(w)$ is the optimal centre.

Proof. "$\ge$" is Proposition 4.1. For "$\le$", take the optimal $B(x, R)$ and the certificate weights w of Proposition 3.3, padded with zeros so that $w \in \Delta_n$. Then $x(w) = x$, and putting $y = x$ in (5) gives $\varphi(w) = \sum_i w_i \, \|p_i - x\|^2 = R^2$, because $w_i > 0$ only for support points, which are at distance exactly R. For the last claim, suppose $\varphi(w) = R^2$; taking $y = x$ in (5) and using $\sum_i w_i \, \|p_i - x\|^2 \le R^2$ forces $\|x(w) - x\|^2 \le 0$. □

Remark. Readers who have met Lagrangian duality will recognise φ as the dual function of (1) and the u_i as the multipliers of the constraints $\|p_i - x\| \le r$. Readers who have not have lost nothing: every statement above was proved with the identity (5) and one line of algebra. The weights have a perfectly concrete meaning — u_i is how hard point i pulls on the centre, and $u_i = 0$ means it is not pulling at all.

4.2 What optimality looks like in the weights

Differentiating (4) and using (5),

$$\frac{\partial \varphi}{\partial u_i}(u) \;=\; \|p_i\|^2 - 2\, x(u)^{\mathsf{T}} p_i \;=\; \|p_i - x(u)\|^2 - \|x(u)\|^2 . \tag{6}$$

The optimality conditions for maximising a concave function over the simplex say that at a maximiser w there is a number λ with

$$\frac{\partial \varphi}{\partial u_i}(w) = \lambda \quad \text{whenever } w_i > 0, \qquad \frac{\partial \varphi}{\partial u_i}(w) \le \lambda \quad \text{whenever } w_i = 0.$$

Substituting (6) and cancelling the common $\|x\|^2$, these read

$$\|p_i - x\| = R \;\text{ for all } i \text{ with } w_i > 0, \qquad \|p_i - x\| \le R \;\text{ for all } i \text{ with } w_i = 0. \tag{7}$$

Which is the geometric certificate, word for word: *the points carrying weight are on the boundary, and everything else is inside.* The optimisation and the geometry are two descriptions of one object, and the algorithm below can be read in either language.

5 The active-set algorithm

Corollary 3.4 says the answer is determined by at most $d+1$ points. If we knew which ones, we could finish in a single linear solve. We do not, so we guess and correct. That is the whole idea.

5.1 The state

The algorithm carries a *working set* (or *support set*) $F \subseteq \{1, \dots, n\}$ of indices — the current guess at which points touch the boundary — together with weights $u_i > 0$ for $i \in F$ summing to one. In the language of Section 4, we are running an active-set method on the QP $\max_{\Delta_n} \varphi$: the variables in F are free and all others are pinned at $u_i = 0$.

5.2 The subproblem: fit a ball to the working set

Fix F with points $q_0, \dots, q_{m-1}$. Ignore the other points entirely and solve the equality-constrained subproblem: maximise φ over weights supported on F and summing to one, with no sign restriction. By the same computation that produced (7) — but now with no inequality multipliers, because there are no inequalities left — a stationary point is characterised by

$$\|q_j - x\| \text{ equal for all } j, \qquad x \in \operatorname{aff}\{q_0, \dots, q_{m-1}\}.$$

So the subproblem is a piece of classical geometry: *find the point in the affine hull of the working set that is equidistant from all of them*, i.e. the circumcentre of the face.

Parametrise the affine hull by the edge vectors $e_j = q_j - q_0$, $j = 1, \dots, m-1$, collected as the rows of the $(m-1) \times d$ matrix D, and write $x = q_0 + D^{\mathsf{T}} y$. Then

$$\|x - q_j\|^2 = \|x - q_0\|^2 \iff -2\, e_j^{\mathsf{T}}(x - q_0) + \|e_j\|^2 = 0 \iff \left(DD^{\mathsf{T}} y\right)_j = \tfrac{1}{2}\,\|e_j\|^2 .$$

This is an $(m-1) \times (m-1)$ system with the Gram matrix DD^{T}, which is positive definite exactly when the edges are linearly independent — that is, when $q_0, \dots, q_{m-1}$ are *affinely independent*. Since $m \le d+1$ in the cases we care about, this is a very small dense solve. Converting back to weights,

$$w = \Big(1 - \textstyle\sum_j y_j,\ y_1,\ \dots,\ y_{m-1}\Big), \qquad x = \textstyle\sum_j w_j q_j. \tag{8}$$

5.3 Reading the signs

The subproblem ignored the constraint $u \ge 0$, so its answer may violate it, and that is exactly the information we need.

- **All $w_j \ge 0$.** The circumcentre lies inside the region spanned by the working set, so half the certificate (7) already holds: the points of F are on the boundary and surround the centre. Accept the step and go check the other half.
- **Some $w_j < 0$.** The circumcentre has fallen *outside* the working set's convex hull — Example 3.5 is precisely this situation, with $w_C = -1$. That point is not holding the ball out; it is being dragged along. Rather than jumping to an infeasible w, move from u towards w and stop the moment the first weight reaches zero:

$$\alpha = \min\Big\{ \frac{-u_i}{s_i} \ :\ s_i < 0 \Big\}, \qquad s = w - u, \qquad u \leftarrow u + \alpha s. \tag{9}$$

Drop the index that hit zero from F and repeat. This is the classical *ratio test* of active-set methods, and here it has a plain geometric reading: eject the point that stopped touching the boundary.

5.4 Growing the working set

Suppose the subproblem gave a feasible w, with centre x and radius R. Sweep *all* n points and compare. If $\|p_i - x\| \le R$ for every i, then by (7) — or directly by Theorem 3.2 — we are done and the answer is exact. Otherwise pick the worst offender,

$$k \;=\; \arg\max_i \|p_i - x\|^2 \,,$$

add k to F with initial weight 0, and repeat. Adding the *farthest* violator is the greedy choice: it is the point in most urgent need of covering, and by (6) it is also the free variable with the largest improvement in φ, so it is steepest ascent as well as common sense.

5.5 The degenerate case

One case remains. The working set can become affinely *dependent* — three collinear points in the plane, or any $d+2$ points — and then the Gram matrix DD^{T} is singular and there is no unique circumcentre to jump to.

Look at what that means for φ. A weight direction s satisfying

$$\sum_i s_i = 0 \qquad \text{and} \qquad \sum_i s_i q_i = 0$$

moves the weights without moving the centre $x(u)$ at all. Eliminating $s_0 = -\sum_{j\ge 1} s_j$ turns the second condition into $D^{\mathsf{T}} s_{1:} = 0$, so such directions exist precisely when D has a non-trivial left null space, i.e. precisely when the face is affinely dependent. Along any of them the term $\|x(u)\|^2$ in (4) is frozen, so

$$\varphi(u + ts) = \varphi(u) + t \sum_i s_i \|p_i\|^2$$

is *linear* in t: the subproblem is unbounded, and the right response is not to jump anywhere but to slide. Project the gradient (6) onto the null space, follow it, and apply the same ratio test (9). A weight is driven to zero, the working set shrinks by one, and affine independence is restored.

5.6 The algorithm, assembled

Active-set method for the smallest enclosing ball

Input: points $p_1, \dots, p_n \in \mathbb{R}^d$. *Output:* R, x, weights u.

1. Recentre: replace p_i by $p_i - \bar{p}$ (undo at the end).
2. Initialise $F = \{k\}$ where k is the point farthest from p_1, and $u_k = 1$.
3. **repeat**
   (a) If the face $\{p_i\}_{i \in F}$ is affinely *dependent*: compute a descent direction s in the null space, take the ratio step (9), drop a zero weight, and continue.
   (b) Otherwise solve (8) for the circumcentre weights w.
   (c) If $\min_j w_j < 0$: set $s = w - u$, take the ratio step (9), drop a zero weight, and continue.
   (d) Otherwise accept $u \leftarrow w$, $x \leftarrow x(u)$, $R \leftarrow \max_{i \in F} \|p_i - x\|$.
   (e) Let $k = \arg\max_i \|p_i - x\|$. If $\|p_k - x\| \le R$: **return** $(R,\ x + \bar{p},\ u)$.
   (f) Otherwise drop the zero weights from F, append k with weight 0.

Example 5.1 (Example 3.5 by hand)**.** Start with $F = \{A, B\}$ (the two farthest-apart points). The circumcentre of two points is their midpoint $(2, 0)$ with weights $(\frac{1}{2}, \frac{1}{2}) \ge 0$, so $R = 2$. Sweep: $\|C - (2,0)\| = \sqrt{2} < 2$, so nothing is outside and we return immediately.

Start instead with $F = \{A, C\}$ to see the interesting branch. The midpoint is $(0.5, 0.5)$ with $R = \sqrt{0.5} \approx 0.707$; the farthest point is B, at distance ≈ 3.54, so B joins and $F = \{A, C, B\}$. The circumcentre of the three is $(2, -1)$ with weights $(1.25, -1, 0.75)$ — negative on C. The ratio test (9) moves as far as it can and drops C, leaving $F = \{A, B\}$, and the next iteration returns $R = 2$, $x = (2, 0)$ as before.

5.7 Why it terminates, and how fast

Termination. Each iteration either strictly increases φ or strictly shrinks the working set, and there are finitely many working sets, so the method cannot wander forever. Two remarks are in order. First, this is the standard finite-termination argument for active-set methods, and like all of them it has a caveat: a degenerate step of length $\alpha = 0$ makes no progress in φ, and in exact arithmetic one needs an anti-cycling rule (Bland's, say) to rule out an infinite loop. In practice implementations use an iteration cap as a safety net and never hit it. Second, when the method stops it stops at an *exact* vertex-like configuration — a specific working set with a specific linear solve — not at a tolerance. On inputs whose answer is exactly representable, it returns that answer bit for bit.

Cost. Per iteration:

- one sweep of squared distances over all points, $O(nd)$, which vectorises perfectly;
- one Gram matrix and dense solve of size $m - 1 \le d$, so $O(m^2 d + m^3)$.

Note what is *not* there: no factorisation of anything of size n. The linear algebra is governed by the ambient dimension, and n enters only through one streaming pass. This is the structural reason the method scales far better in n than a general conic solver, which must factor an n-row constraint matrix at every interior-point iteration. Empirically the

number of iterations is a small multiple of the final support size; the classical worst-case analyses and the expected-time results for randomised variants are in the references (§10).

6 Making it numerically honest

The mathematics above is exact. Floating point is not, and a naive transcription fails on inputs that are geometrically unremarkable. Four points are worth knowing; all four are consequences of one principle: *no tolerance may secretly assume that the coordinates are of size* 1.

Recentre first. Replace p_i by $p_i - \bar{p}$ before doing anything. Otherwise consider a cloud of extent 1 sitting a million units from the origin: the squared coordinates are of size 10^{12}, their rounding error is of size $10^{12}\varepsilon \approx 10^{-4}$, and the radius we are trying to resolve is of size 1. Every quantity in the algorithm — Gram matrix, distances, rounding floor — should be governed by the *extent* of the cloud, never by its distance from an arbitrary origin. Recentring costs one pass and buys exactly that. The result is that the answer is invariant under translation of the input, as of course it must be mathematically.

Difference before you square. Compute $\|p - x\|^2$ as $\sum_k (p_k - x_k)^2$, never as $\|p\|^2 - 2p^{\mathsf{T}}x + \|x\|^2$. The second form subtracts quantities of size $\|x\|^2$ from each other to produce an answer of size R^2, and every digit by which $\|x\|$ exceeds R is a digit lost to cancellation. The first form has no cancellation at all.

Test rank on the edges. The affine-dependence test of Section 5 is a rank test. It is tempting to run it on the stacked matrix $\begin{pmatrix} \mathbf{1}^{\mathsf{T}} \\ Q^{\mathsf{T}} \end{pmatrix}$, which encodes "sums to zero and fixes the centre" in one go. Do not: the row of ones has entries of size 1, so it dominates the singular values, and any cloud whose extent is small relative to 1 is then declared degenerate whatever its actual geometry. Eliminating the sum constraint by hand and testing the rank of the edge matrix D — as in Section 5 — keeps the test scale-invariant.

Size the tolerances in the right units. The stopping test "is any point outside?" compares squared distances, which carry units of length^2. A relative tolerance handles the ordinary case; but near the degenerate limit one also needs an absolute floor, and that floor must be proportional to $\varepsilon \max_i \|p_i\|^2$ rather than a fixed constant. The same applies to the null-space direction of the degenerate branch: the gradient carries units of length^2, so normalise the direction to unit maximum before comparing anything derived from it against a dimensionless weight tolerance. Otherwise, on a cloud of extent 10^{-20}, no component ever looks binding.

Both invariances — under translation and under scaling — deserve explicit regression tests, since they are easy to break with a well-meant simplification and produce plausible-looking wrong answers rather than crashes.

7 Using the code

These notes accompany `cvxball`, which ships the active-set method of Section 5 and nothing else: NumPy is its only dependency. The second-order cone program of Section 4 is implemented too, but under `experiments/`, where it is what it has in fact become —

the reference the shipped method is measured against, sharing its interface so the two can be tested against each other.

```
import numpy as np
from cvxball import min_circle_active_set
from experiments.clarabel_ball import min_circle_clarabel # reference

points = np.array([[0.0, 0.0], [4.0, 0.0], [1.0, 1.0]])

radius, centre = min_circle_active_set(points) # -> 2.0, array([2., 0.])
radius, centre = min_circle_clarabel(points) # the same, to solver tolerance
```

The correspondence with the text is direct:

In these notes	In the source
Subproblem (8), the circumcentre	`_face_weights`
Affine-dependence test and null space	`_affine_null_space`
Squared distances, differenced	`_sq_dist`
The main loop	`min_circle_active_set`
The cone program of §4	`experiments/clarabel_ball.py`
Welzl's algorithm, for comparison	`experiments/welzl.py`
Conic front end, §8	`cvxball.active_set.DefaultSolver`
Goldfarb–Idnani QP, §8.3	`cvxball.qp.solve_qp`

A third entry point, `cvxball.active_set`, wraps the same method in the interface of a general-purpose solver; that is the subject of Section 8.

The optimal weights u of Theorem 4.2 are more than a by-product: they are the certificate. Given (R, x, u) a sceptic can verify optimality without re-running anything — check $u \geq 0$, check $\sum_i u_i = 1$, check $x = \sum_i u_i p_i$, check $\|p_i - x\| \leq R$ for all i with equality wherever $u_i > 0$ — and Theorem 3.2 then guarantees the answer. Verifying a solution is cheaper than finding one, which is a good habit to acquire early.

8 The same idea in a solver's clothing

Everything so far has been about one specific problem. This last section shows what happens when you package the method as a *solver* — something that accepts a problem in a standard form rather than a point cloud — and what the weights of Theorem 4.2 turn into when you do. Nothing new is proved here; the point is that the objects we built by hand are the objects the general theory already had names for.

8.1 Conic standard form

A conic program is written

$$\text{minimise} \quad \tfrac{1}{2} z^{\mathsf{T}} P z + q^{\mathsf{T}} z \qquad \text{subject to} \qquad Az + s = b, \quad s \in \mathcal{K}, \tag{10}$$

where $\mathcal{K}$ is a product of simple convex cones. Three kinds cover most of what one meets:

Cone	Definition	What it expresses
zero cone	$\{0\}$	an equality row $a^{\mathsf{T}} z = b$
non-negative orthant	$\{s : s \geq 0\}$	an inequality row $a^{\mathsf{T}} z \leq b$
second-order cone $\mathcal{Q}^{k+1}$	$\{(t, v) \in \mathbb{R} \times \mathbb{R}^k : \|v\| \leq t\}$	a Euclidean norm bound

The slack variable s is what the constraint leaves over, and requiring $s \in \mathcal{K}$ is what makes it a constraint. Our problem (1) fits with no cleverness at all. Take the decision vector $z = (r, x_1, \ldots, x_d)$, minimise r (so $P = 0$ and q is the first unit vector), and note that

$$\|p_i - x\| \le r \quad \Longleftrightarrow \quad s_i := (r,\ p_i - x) \in \mathcal{Q}^{d+1}.$$

One second-order cone per point, each of dimension $d+1$, stacked into one A and one b. That is the whole translation, and it is what `_build_soc_program` performs.

8.2 The weights, in the solver's coordinates

A conic solver is expected to return not just the primal z but a *dual* vector ζ, one block per cone, certifying optimality. For the ball problem we already have a certificate — the weights u — so the two had better be the same object. They are, after one change of coordinates: point i's block is

$$\zeta_i \ = \ u_i \left(1,\ -\frac{p_i - x}{R}\right). \tag{11}$$

Two lines of arithmetic show this is exactly what the theory asks for.

It lies in the cone. The tail of ζ_i has norm $u_i \|p_i - x\| / R$, which for a support point ($\|p_i - x\| = R$) equals u_i, its head. So ζ_i sits on the *boundary* of $\mathcal{Q}^{d+1}$ for every support point, and is the zero vector for every other point.

It is complementary to the slack. Pair ζ_i with the slack $s_i = (R,\ p_i - x)$ of Section 8.1:

$$s_i^{\mathsf{T}} \zeta_i \ = \ u_i \left(R - \frac{\|p_i - x\|^2}{R}\right) \ = \ \frac{u_i}{R}\Big(R^2 - \|p_i - x\|^2\Big) \ = \ 0,$$

because each factor kills the other: where $u_i > 0$ the point is on the boundary and the bracket vanishes; where the bracket does not vanish, $u_i = 0$. This is *complementary slackness*, and comparing it with (7) shows it is the certificate of Section 3 written in the solver's alphabet. The geometry (*which points touch*), the optimisation (*which multipliers are non-zero*) and the conic algebra (*which blocks are on the cone boundary*) are three vocabularies for one fact.

Remark. This is why an implementation should keep the weights rather than discard them. Returning (R, x) answers the question; returning (R, x, u) lets the caller *check* the answer, and — via (11) — hands a conic solver's caller the dual it expects, at no extra cost.

8.3 The general quadratic program

Once the front door accepts (10), most incoming problems are not enclosing balls. If every cone is a zero cone or an orthant, (10) is an ordinary convex quadratic program,

$$\text{minimise} \quad \tfrac{1}{2} x^{\mathsf{T}} P x + q^{\mathsf{T}} x \qquad \text{subject to} \qquad Ex = f, \quad Gx \le h, \tag{12}$$

with P symmetric positive definite. The same active-set idea applies, in the form given by Goldfarb and Idnani [6], and it is instructive to see how little changes:

- **Working set.** A guess at which of the inequality rows of G are tight at the optimum — the analogue of the support set.
- **Subproblem.** Minimise the objective subject to the working-set rows holding as *equalities*. That is one $k \times k$ dense solve with k the size of the working set — the analogue of (8).

- **Drop.** If a multiplier would go negative, that row is not really pushing; ratio-test to the boundary and drop it — the analogue of (9).
- **Add.** Otherwise scan all the rows and add the most violated one — the analogue of adding the farthest point.

The one genuinely new idea is the choice of the word *dual* in "dual active-set method". A *primal* method must start from a feasible point, and finding one is a separate optimisation problem in its own right ("phase one"). Goldfarb–Idnani sidesteps this: it starts at the unconstrained minimiser $x = -P^{-1}q$, which is always available and needs only one Cholesky factorisation, and it adds constraints one at a time while keeping every multiplier non-negative. Feasibility is what it works *towards*; optimality of the current working set is the invariant it never gives up. This is why the enclosing-ball method of Section 5 could also begin anywhere it liked — with a single point — and still be correct at every step.

Remark (On not answering a different question). The method above needs $P \succ 0$; the subproblem is otherwise not a minimum. A tempting shortcut is to add ϵI to any P until it factorises. For a P that is positive semidefinite but numerically borderline that is a fair regularisation. For a genuinely *indefinite* P it is not: the true problem has no minimum, and $P+\epsilon I$ silently replaces it with a convexified one whose answer is unrelated. Refusing is the honest response, and a solver that returns a confident number for an ill-posed question is worse than one that declines.

8.4 Why bother, when interior-point methods exist

Interior-point methods solve (10) well and are the right tool for large sparse programs. The trade the active-set methods offer is different, and worth stating plainly because it is the reason this package exists.

An interior-point method approaches the optimal face *from the inside*. It stops when a duality gap falls below a tolerance, at which point the constraints that are "really" tight have slacks of perhaps 10^{-9} and the ones that are not have slacks of 10^{-4}, and the caller must decide where to draw the line. An active-set method identifies the face *combinatorially* — a set of indices — and then solves that face exactly. If your question is *which* constraints bind (which holdings sit on their bound, which points touch the sphere), one method returns a discrete answer and the other returns numbers you must round. Per iteration the active-set method also factors nothing of size n, which is what makes it scale in the number of constraints the way Section 5 described.

Remark (A note on defensive programming). Recognising "an enclosing-ball program" by counting cones is not enough: a program can have n second-order cones of dimension $d+1$ and still be some other problem entirely. The safe check is to reconstruct the point cloud from (A, b), rebuild the program those points *would* produce, and compare it with the one that arrived. A look-alike is then refused rather than answered with the solution to a problem nobody asked about.

9 Exercises

Exercise 1. Verify the claims of Example 3.5: that the circle through $(0,0), (4,0), (1,1)$ has centre $(2,-1)$ and radius $\sqrt{5}$, and that the barycentric weights of that centre with respect to the three points are $(1.25,\ 0.75,\ -1)$. Which sign told you the circumcircle was not the answer?

Exercise 2. Prove that the smallest enclosing ball of S equals the smallest enclosing ball of the vertices of $\operatorname{conv} S$. (Hence no interior point is ever a support point.)

Exercise 3. Show that in the plane the support set of the optimal ball never consists of three points forming an obtuse triangle. Generalise: if three support points form a triangle with an obtuse angle at q_j, derive a contradiction with $x \in \operatorname{conv} T$.

Exercise 4. Fill in the algebra of (5), and deduce Lemma 2.2 from it by choosing $n = 2$ and $u = (\tfrac{1}{2}, \tfrac{1}{2})$ applied to the two *centres*. (The shrinking lemma and the parallel-axis identity are the same fact.)

Exercise 5. Let S lie on a sphere of radius ρ about the origin. Show $\varphi(u) = \rho^2 - \|x(u)\|^2$, so that maximising φ is the same as finding the point of $\operatorname{conv} S$ closest to the origin. Interpret the case $0 \notin \operatorname{conv} S$.

Exercise 6. Implement the algorithm from the box in Section 5 in fifty lines, ignoring all of Section 6. Then construct an input on which it fails: translate a unit-extent cloud far from the origin, and separately shrink one to extent 10^{-20}. Explain each failure in terms of the four principles listed there.

Exercise 7. Verify directly that the vector ζ_i of (11) satisfies $\|\text{tail}\| \le \text{head}$ for *every* i, not merely for the support points, so that ζ really is a point of the cone. Where in the argument did you use $\|p_i - x\| \le R$?

Exercise 8. Write the enclosing-ball problem in the form (12) instead — that is, as a quadratic program in the weights u over the simplex, using Theorem 4.2. What are P, q, E, f, G and h? Is your P positive definite, and what does your answer mean for Section 8.3?

Exercise 9. The method as described adds one point per outer iteration. Suppose instead you add the k farthest violators at once. Does the algorithm still terminate? Does the certificate check change? (This is the "multiple pricing" idea from linear programming.)

10 Notes and further reading

The problem is old: Sylvester posed the planar case in 1857 [8]. Elzinga and Hearn [2] gave an early finite algorithm of the type developed here. Welzl's randomised incremental algorithm [9] runs in expected $O(n)$ time for fixed d and is beautifully short, though its recursion degrades as d grows. Gärtner [4] made it robust.

From there the subject forks, and it is worth being clear about which branch this is. The method of Section 5 is of the quadratic-programming line — Gärtner and Schönherr [5] — whose state is the weight vector of Section 4 and whose every pivot solves the face exactly. Fischer, Gärtner and Kutz [3] take the other branch: they keep a pair (support set, centre) and *walk* the centre towards the circumcentre, stopping the moment a new point catches the shrinking sphere, so their iterate is in general not a circumcentre at all. They also supply what Section 5 only gestured at — an explicit Bland-type order on the points, with a proof that it terminates — and then note that it is too slow to use in practice. Their reservation about QP formulations, that robustness past a few hundred dimensions demanded exact arithmetic, is worth carrying alongside Section 6: the invariances there are an answer to that concern, not evidence against it.

The active-set machinery — working sets, ratio tests, dropping constraints — is standard quadratic programming; Goldfarb and Idnani [6] is the canonical dual method and

Nocedal and Wright [7] the standard textbook treatment. For the convex-analytic background, including conic formulations and duality done properly, see Boyd and Vandenberghe [1], which is freely available.

A last word on what this problem teaches. Almost every idea here recurs: a certificate that is cheaper to check than to find; an optimality condition that reads simultaneously as geometry and as calculus; an algorithm that guesses the combinatorial structure and repairs it; and a numerical implementation whose tolerances must be dimensionally honest. The smallest enclosing ball is small enough to hold in your head and rich enough to contain all four.

References

[1] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[2] J. Elzinga and D. W. Hearn. Geometrical solutions for some minimax location problems. *Transportation Science* 6(4):379–394, 1972.

[3] K. Fischer, B. Gärtner and M. Kutz. Fast smallest-enclosing-ball computation in high dimensions. In *Proc. 11th European Symposium on Algorithms (ESA)*, 630–641, 2003.

[4] B. Gärtner. Fast and robust smallest enclosing balls. In *Proc. 7th European Symposium on Algorithms (ESA)*, 325–338, 1999.

[5] B. Gärtner and S. Schönherr. An efficient, exact, and generic quadratic programming solver for geometric optimization. In *Proc. 16th Annual ACM Symposium on Computational Geometry*, 110–118, 2000.

[6] D. Goldfarb and A. Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming* 27:1–33, 1983.

[7] J. Nocedal and S. J. Wright. *Numerical Optimization*, 2nd edition. Springer, 2006.

[8] J. J. Sylvester. A question in the geometry of situation. *Quarterly Journal of Pure and Applied Mathematics* 1:79, 1857.

[9] E. Welzl. Smallest enclosing disks (balls and ellipsoids). In *New Results and New Trends in Computer Science*, LNCS 555, 359–370, 1991.